

Code-documentatie

De Visdeurbel-datavisualisatie

De Mitchell-pagina, stap voor stap uitgelegd
React-componenten & JavaScript-grafieken

Een begrijpelijke gids door alle actieve bestanden in
`src/components/mitchell-components/` en `src/scripts/mitchell/charts/`

Inhoudsopgave

1. Inleiding — voor wie en hoe te lezen
2. Het grote plaatje — hoe de pagina werkt
3. De pagina zelf — `Mitchell.jsx`
4. De dirigent — `mittchell.js`
5. De gedeelde fundering (data & hulpjes)
6. Structurele componenten (FishSprite, DataSwitch, SectionWave)
7. De feiten-carrousel — `DataCarousel.jsx`
8. Het aquarium — `Aquarium.jsx` + `aquarium.js`
9. De sonar/radar — `RadarChapter.jsx` + `radar.js`
10. Het talenkoor — `LanguagesChapter.jsx` + `languages.js`
11. Het net — `NetChapter.jsx` + `net.js`
12. Begrippenlijst (jargon uitgelegd)
13. Bijlage — de legacy-map

1 · Inleiding — voor wie en hoe te lezen

Dit document legt uit hoe de scrollende datavisualisatie over de Visdeurbel in elkaar zit — bedoeld zodat **iedereen** het kan volgen, ook zonder diepe programmeerkennis.

De pagina bestaat uit twee soorten bestanden die samenwerken:

Soort	Wat doet het?	Map
React-componenten .JSX	Bouwen de structuur van de pagina: de teksten, knoppen en lege "podia" (stages) waar straks een grafiek in komt. Vergelijk het met het neerzetten van lege lijsten aan de muur.	components/mitchell-components/
Chart-scripts .JS	Tekenen en animeren de grafieken in die podia, met data en de tekenbibliotheek D3 of een canvas. Dit is het "schilderij" dat in de lijst komt.	scripts/mitchell/charts/

Leeswijzer. Per grafiek bespreken we eerst het korte React-component (de lege huls) en daarna het script dat de huls vult. Jargon (D3, canvas, IntersectionObserver...) staat uitgelegd in de **Begrippenlijst** achterin — bij het eerste gebruik markeren we zo'n term *schuin*.

2 · Het grote plaatje — hoe de pagina werkt

De Mitchell-pagina is één lange scroll met "hoofdstukken" (chapters). Het samenspel verloopt in vier stappen:

- 1 **React zet de structuur neer.** `Mitchell.jsx` rendert alle hoofdstukken als HTML. Elk grafiek-hoofdstuk bevat een lege `<div>` met een uniek `id` — bijvoorbeeld `id="aquariumStage"`. Dat is het lege podium.
- 2 **De dirigent start.** Eén keer bij het laden roept de pagina `initMitchell()` aan (in `mitchell.js`). Die laadt de data uit een JSON-bestand en houdt in de gaten welk hoofdstuk in beeld scrolt.
- 3 **Grafiek wordt "lui" getekend.** Zodra een hoofdstuk in beeld komt, roept de dirigent het bijbehorende script aan (bijv. `initAquarium()`), dat het podium vult. Dit heet *lazy loading*: pas tekenen wanneer nodig, dat scheelt rekenkracht.
- 4 **Wisselen van periode.** Bovenaan staat een schakelaar (week / maand / jaar). Klik je daarop, dan worden alle grafieken opgeruimd en opnieuw getekend met de nieuwe dataset.

Waarom deze splitsing? React is sterk in *structuur en toestand* (welke knop is actief, welke tekst staat er). D3/canvas zijn sterk in *tekenen en animeren*. Door ze te scheiden blijft elk stuk overzichtelijk: de `.jsx` blijft kort, en alle teken-logica zit gebundeld in de `.js`-scripts.

De gedeelde "waarheid": `state`

Alle grafieken lezen uit één gedeeld data-object, `state`. De dirigent vult dat met de geladen dataset; de grafieken halen er hun cijfers uit. Zo hoeft bij een periodewissel maar op één plek de data ververs te worden.

3 · De pagina zelf — `Mitchell.jsx` `.JSX`

`src/pages/Mitchell.jsx`

Dit is het hoofdcomponent van de pagina. Het regelt drie dingen: stylesheets laden, de dirigent starten, en de hoofdstukken in de juiste volgorde neerzetten.

Stap voor stap

- 1 **Stylesheets inladen.** Met `useStylesheet('/styles/mitchell.css')` (en enkele andere) worden de bijbehorende CSS-bestanden alleen op deze pagina geladen.
- 2 **Dirigent één keer starten.** In een `useEffect` wordt `initMitchell()` aangeroepen. Een `useRef` (`initialized`) zorgt dat dit maar één keer gebeurt, ook als React in ontwikkelmodus dubbel rendert. De functie geeft een *opruiim-functie* terug die draait als je de pagina verlaat.
- 3 **De hoofdstukken plaatsen.** In de `return` staan de secties in scroll-volgorde, met tussen elke twee een `<SectionWave>` (de golf-overgang):

```
// vereenvoudigde volgorde uit de return
<DataCarousel />           // feiten-carrousel
<SectionWave ... />
<Aquarium />               // rondzwemmende vissen
<SectionWave ... />
<RadarChapter />           // sonar
<SectionWave ... />
<LanguagesChapter />       // drijvende begroetingen
<SectionWave ... />
<NetChapter />             // bubbel-net
```

Let op — Hero, Ring en Wereldkaart staan uit. In de code zie je `<HeroChapter />`, `<RingChapter />` en de wereldkaart *uitgecommentarieerd* (tussen `{/* ... */}`). Ze worden dus niet getoond; hun code is naar de `legacy/`-map verhuisd. Daarom bespreken we ze hier niet (zie de bijlage).

Het toegankelijkheidsmenu (`AccessibilityMenu`)

Binnen hetzelfde bestand zit een klein menu (het -knopje) met twee schakelaars:

- **Animaties uit** — zet `window.__reduceMotion` aan. Alle grafieken checken die vlag en slaan dan animaties over (zie `reduceMotion()` in de fundering).
- **Hoger contrast** — voegt een class toe aan de `<body>` waarop de CSS reageert.

Beide keuzes worden in `localStorage` bewaard, zodat ze bij een volgend bezoek onthouden blijven.

4 · De dirigent — `mittchell.js` `.JS`

`src/scripts/mittchell.js`

Dit script verbindt de lege podia met de juiste grafiek-scripts en regelt het laden en wisselen van data.

Stap voor stap

- 1 **Koppeltabel.** Een object `chapterInit` koppelt elk sectie-`id` aan een teken-functie, bijv. `'ch-radar': initRadar`.
- 2 **In de gaten houden.** Een `IntersectionObserver` let op welke `.chapter` in beeld scrolt. Komt er één in zicht én is die nog niet getekend (`dataset.inited`), dan draait de bijbehorende init-functie. Dit is de *lazy loading* uit stap 3.
- 3 **Opruimen.** `clearChapters()` draait alle geregistreerde opruim-functies (`cleanups`), stopt lopende animaties (`rafs`) en leegt de podia. Nodig vóór een periodewissel, anders stapelen grafieken en animaties zich op.
- 4 **Periode wisselen.** `setPeriod('week'|'maand'|'jaar')` markeert de actieve knop, ruimt op, laadt het juiste JSON-bestand (en voor 'jaar' wordt de data eerst gesynthetiseerd) en begint opnieuw te observeren.
- 5 **Opstarten.** `boot()` laadt de begindataset (maand) + de wereldkaart, koppelt de schakelknoppen, en start het meescrollende gids-visje.

```
// het hart van de lazy-init
const sectionObserver = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (!entry.isIntersecting) return; // nog niet in beeld → niks doen
    if (!entry.target.dataset.inited) { // nog niet getekend?
      entry.target.dataset.inited = '1';
      chapterInit[entry.target.id]?.(entry.target); // teken de grafiek
    }
  });
});
```

Levenscyclus (lifecycle). Elke grafiek "registreert" zijn opruim-werk (timers, observers, animaties) in een gedeelde lijst. Zo kan de dirigent bij een wissel of bij het verlaten van de pagina alles netjes stoppen — geen geheugenlekken of spookanimaties.

5 · De gedeelde fundering (data & hulpjes)

Zeven kleine modules die alle grafieken gebruiken. Kort uitgelegd:

`constants.js` — de vaste gegevens

Pure data, geen logica: het kleurenpalet (`COLORS`), lettertypes, maandnamen, de lijst **vissoorten** (`visData` — met naam, gewicht, typische lengte, vorm, kleur), landcoördinaten voor de kaart, browserfamilies en begroetingen per taal.

`state.js` — de gedeelde waarheid

Eén object `state` met de actieve dataset (totalen, uren per dag, periode-label...). Plus `lifecycle` (opruimlijst) en `raf()`, een wrapper om `requestAnimationFrame` die het animatieframe registreert zodat het later gestopt kan worden.

`utils.js` — kleine gereedschappen

- `$` en `$$` — korte manieren om één of meerdere HTML-elementen op te zoeken.
- `formatNumber` — toont getallen netjes met Nederlandse punten (49736 → "49.736").
- `reduceMotion()` — waar/onwaar of animaties uit moeten.
- `mulberry32 / rng` — een *seeded* toevalsgenerator: met dezelfde startwaarde altijd dezelfde "willekeur", zodat posities bij elke render gelijk zijn.

`dataLoad.js` — data ophalen & jaar verzinnen

`loadData(url)` haalt een JSON-bestand op en kopieert alles naar `state`. `synthesizeYear()` blaast de maand-snapshot op tot een heel jaar (de echte jaardata is te groot voor de browser), met een zachte seizoensgolf.

`tooltip.js` — één gedeelde tooltip

`showTooltip(html, x, y)` en `hideTooltip()` vullen en positioneren één enkel tooltip-element dat de hele pagina deelt — efficiënter dan per grafiek een eigen tooltip.

`fishImage.js` — vis-foto's inkleuren

De vis-PNG's zijn neutraal grijs. `ensureTintFilter()` maakt een SVG-filter dat de foto eerst grijs maakt en daarna met de soort-kleur vermenigvuldigt — zo houden de vissen hun textuur maar krijgen ze hun eigen kleur (*luminantie-behoudend tinten*).

`swimFish.js` — het meescrollende gids-visje

Het kleine visje (het baars-plaatje) dat met je meescrollt. Elke frame zoekt het de grafiek die het dichtst bij het midden van het scherm staat en cirkelt daar soepel omheen met de animatiebibliotheek GSAP. Buiten beeld dobbert het rechtsonder rond.

6 · Structurele componenten

Drie kleine React-componenten zonder eigen grafiek-script.

FishSprite.jsx .JSX

components/mitchell-components/FishSprite.jsx

Een onzichtbare SVG (`width="0"`) met een set herbruikbare vis-silhouetten (`<symbol>`): rond, baars, roofvis, lang, klein. Andere grafieken kunnen zo'n silhouet aanroepen i.p.v. het telkens opnieuw te tekenen. Met `fill="currentColor"` neemt elk silhouet de kleur over die de aanroeper instelt.

DataSwitch.jsx .JSX

components/mitchell-components/DataSwitch.jsx

De drie knoppen **Week · Maand · Jaar** bovenaan. Het component levert alleen de knoppen met een `data-period`-attribuut; de werking (klik → andere dataset) zit in `mitchell.js` (`setPeriod`). "Maand" staat standaard actief.

SectionWave.jsx .JSX

components/mitchell-components/SectionWave.jsx

De golvende overgang tussen twee secties. Krijgt twee kleuren mee: `top` (sectie erboven) en `bottom` (eronder). De golf-vorm is een breed, periodiek SVG-pad dat met een subtiele CSS-animatie horizontaal driftt, zodat het naadloos doorloopt.

7 · De feiten-carrousel — DataCarousel.jsx .JSX

components/mitchell-components/DataCarousel.jsx

Een doorbladerbare reeks "Wist je dat?"-kaartjes. Dit is een **volledig React-component** (geen los chart-script) dat de data zelf inleest en omzet naar feiten.

Stap voor stap

- 1 **Data ophalen.** In een `useEffect` wordt `/json/vis-data.json` opgehaald. De vlag `alive` voorkomt dat de data nog wordt gezet als het component intussen verdwenen is.
- 2 **Feiten bouwen.** `buildFacts(d)` rekent uit de ruwe data zeven feiten uit, bijvoorbeeld: het drukste uur (alle uren over de dagen opgeteld), het meest gespotte vissoort, het percentage mobiele bezoekers, en het topland.
- 3 **Carrousel-techniek.** De bibliotheek *Embla* (`useEmblaCarousel`) verzorgt het vegen/scrollen. Bij elke wissel onthoudt het component welk kaartje actief is en of de vorige/volgende-pijl aan moet.
- 4 **Toelichting (?).** Een `?`-knop toont per feit een korte bronvermelding (`note`).

```
// voorbeeld van één afgeleid feit
const byHour = new Array(24).fill(0);
(d.weekHours || []).forEach((v, i) => { byHour[i % 24] += v; }); // tel per uur op
const busyHour = byHour.indexOf(Math.max(...byHour));           // drukste uur
```

De highlights in de zinnen (de paarse cijfers) komen uit het kleine hulpje `hl(x)`, dat een stukje tekst vetgedrukt en gekleurd maakt.

8 · Het aquarium — Aquarium.jsx + aquarium.js

components/mitchell-components/Aquarium.jsx · scripts/mitchell/charts/aquarium.js

Het component .JSX

Heel kort: een titel, een lege `<div id="aquariumStage">` (het podium), een rij met filter-knoppen (`#aquariumFilters`) en een tip. Verder niets — het script vult de rest.

Het script .JS — stap voor stap

Dit is de enige grafiek op een *canvas* (i.p.v. SVG), omdat er tientallen visjes tegelijk bewegen — canvas is daarvoor sneller.

- 1 **Canvas opzetten.** Een `<canvas>` wordt in het podium gezet en op scherm scherpte (*device pixel ratio*) afgesteld, zodat het niet wazig is.
- 2 **Steekproef van ~80 vissen.** Per soort wordt het aantal visjes bepaald in verhouding tot hoe vaak die soort gezien is (`v.count / total × 80`). Vaker gezien = meer visjes.
- 3 **Grootte op schaal.** Elk visje krijgt een grootte op basis van het gewicht: $\sqrt[3]{\text{gewicht}}$ (derdemachtswortel), want massa schaalt met lengte³. Zo staan de soorten in realistische verhouding (meerval groot, alver klein). Ook krijgt elk visje een eigen lengte in cm ($\pm$ wat variatie) voor de hover.
- 4 **Visjes inkleuren (sprite).** `makeSprite()` tekent de vis-PNG op een onzichtbaar mini-canvas en loopt pixel voor pixel langs om hem in de soort-kleur te zetten (grijswaarde $\times$ kleur), met behoud van textuur. Het resultaat wordt gecacheet, zodat dit maar één keer per soort+kleur gebeurt.
- 5 **De animatielus.** Elke frame (`requestAnimationFrame`) beweegt elk visje: een beetje koers, een zachte golf, lichte aantrekking naar het midden, een vleugje toeval ("dwalen") en wrijving (afremmen). Aan de rand zwemt het er aan de andere kant weer in.
- 6 **Interactie. Klik** = de visjes schrikken (ze krijgen een duwtje weg van de muis) + een rimpel (`aquarium-rip`). **Hover** = tooltip met naam en grootte (bijv. "Baars ~32 cm"). De filter-knoppen onderaan zetten soorten aan/uit.
- 7 **Teller.** Een getal telt soepel op naar het totaal aantal waarnemingen (met een ease-curve).
- 8 **Zuinig.** Een *IntersectionObserver* start de animatie pas als het aquarium in beeld is en stopt 'm zodra het weg scrolt.

```
// kern van de zwem-natuurkunde, per visje, per frame
f.x += f.vx; f.y += f.vy + Math.sin(f.wiggle) * 0.04; // koers + golf
f.vx += (50 - f.x) * 0.00006; // zachte trek naar midden
f.vx *= 0.992; // wrijving (afremmen)
```

9 · De sonar/radar — RadarChapter.jsx + radar.js

components/mitchell-components/RadarChapter.jsx · scripts/mitchell/charts/radar.js

Het component `.JSX`

Een titel en een vierkant podium `#radarStage` met daarin een leeg detailpaneel `#radarDetail`. Het script tekent de radar en zet er een tijdslijder onder.

Het idee

Een sonar-scherm: elke vissoort is een "ping". **Hoe vaker gezien, hoe dichterbij het midden.** Een draaiende straal (sweep) veegt rond en licht de pings op die hij passeert.

Het script `.JS` — stap voor stap

- 1 **Decor.** Concentrische ringen, een dradenkruis, een subtiele schijf-vulling en de draaiende sweep-wig (een kleurverloop) worden als SVG getekend.
- 2 **Pings plaatsen.** De afstand tot het midden volgt uit het aantal waarnemingen via een *wortelschaal* (`scaleSqrt`). De hoek is willekeurig, maar via *rejection sampling* (gewoon opnieuw proberen tot het past) komen soorten niet bovenop elkaar. Per ping onthouden we de vaste hoek, zodat hij later alleen radiaal beweegt.
- 3 **Naglooi (afterglow).** In de animatielus onthoudt elke ping wanneer de sweep er voor het laatst overheen kwam (`litAt`). Daarna zakt de helderheid in $\sim 1,6$ s weg naar een lage vloer. Zo "tast" de radar bij elke draai opnieuw af in plaats van alles vast aan te laten staan.
- 4 **De tijdslijder (scrubber).** Onder de radar staat een schuif. De tijdvakken hangen af van de periode:
 - week → per dag
 - maand → per week
 - jaar → per maandStand 0 = de hele periode (de totalen).
- 5 **Verdeling per tijdvak.** Er bestaat geen kant-en-klare data per soort per tijdvak, dus die wordt *afgeleid*: de echte drukte-vorm (belletjes per dag) als basis, met een vaste variatie per soort eroverheen — zó genormaliseerd dat elke soort over de tijdvakken exact optelt tot zijn totaal.
- 6 **Relatief positioneren.** Bij elk tijdvak wordt de afstandschaal opnieuw genormaliseerd op de drukste soort van dát moment. Daardoor komt de drukste vis naar het midden en staan ze altijd ten opzichte van elkaar — niet allemaal samengeperst aan de rand.
- 7 **Info op hover.** Beweeg over een ping → het detailpaneel toont naam, aantal, piekmaand, diepte en gewicht; de vis licht op met een kader. Ga je eraf, dan verdwijnt het paneel weer. Werkt ook met toetsenbord (focus).

```
// naglooi: helderheid op basis van tijd sinds de sweep passeerde
const age = (now - p.litAt) / GLOW_MS;
const glow = age >= 1 ? 0 : (1 - age) * (1 - age); // zacht uitdovend
op = FL00R + (1 - FL00R) * glow;                  // nooit helemaal weg
```

Bewuste keuze. De afstand is *relatief per tijdvak*: een vis dicht bij het midden in een rustige week kan absoluut gezien minder waarnemingen hebben dan een vis aan de rand in een drukke week. Het exacte aantal staat altijd in het label naast de vis.

10 · Het talenkoor — LanguagesChapter.jsx + languages.js

components/mitchell-components/LanguagesChapter.jsx · scripts/mitchell/charts/languages.js

Het component `.JSX`

Een titel en een leeg podium `#langStage`. Verder vult het script alles.

Het idee

Elke taal is een zwevend woord — een begroeting (bijv. "Vis", "Fish", "魚"). Hoe groter het woord, hoe meer bezoekers die taal spraken.

Het script `.JS` — stap voor stap

- 1 Top 24 talen.** De talen worden op aantal bezoekers gesorteerd; de begroeting per taal komt uit een tabel (`GREETINGS`).
- 2 Lettergrootte.** Een `scaleSqrt` (15–70px) zet het bezoekersaantal om naar tekstgrootte. Wortelschaal = de **oppervlakte** schaalt eerlijk mee, niet de hoogte.
- 3 Uit elkaar duwen.** Een *force-simulatie* van D3 zorgt dat de woorden netjes verdeeld worden: een lichte trek naar het midden (`forceX / forceY`), een afstotende kracht (`charge`) en een botsing-kracht (`collide`) die overlap voorkomt.
- 4 Dobberen.** Zodra de simulatie tot rust komt, krijgt elk woord een zachte, eindeloze dobber-beweging (een sinus rond zijn rustplek).
- 5 Toegankelijk & interactief.** Elk woord heeft een `aria-label` ("Japans: 1.234 bezoekers") zodat voorleessoftware het als één eenheid voorleest, en toont bij hover een tooltip met percentage.

```
// de vier krachten die de woorden ordenen
d3.forceSimulation(list)
  .force('x', d3.forceX(centerX).strength(0.04)) // naar het midden
  .force('charge', d3.forceManyBody().strength(-4)) // licht afstoten
  .force('collide', d3.forceCollide(d => d.rad)); // niet overlappen
```

11 · Het net — NetChapter.jsx + net.js

components/mitchell-components/NetChapter.jsx · scripts/mitchell/charts/net.js

Het component `.JSX`

Titel, een schakelaar met drie knoppen (**Aantal** · **Biomassa** · **Gewicht/vis**), een podium `#netStage` en een info-regel `#netInfo`.

Het idee

Alle soorten als bellen in een net. De grootte van een bel hangt af van de gekozen weergave: het **aantal**, het **gemiddeld gewicht**, of de **biomassa** (aantal × gewicht).

Het script `.JS` — stap voor stap

- 1 **Het net.** Bovenaan zakt een net-patroon (gebogen SVG-lijnen) in beeld met een korte animatie.
- 2 **Bellen rangschikken.** *Circle packing* (`d3.pack`) stopt de cirkels — op grootte gesond — zo compact mogelijk in het vlak.
- 3 **Drie weergaven.** Een functie per weergave bepaalt de belgrootte: `count` , `weight` , of `count × weight` (biomassa).
- 4 **Morphen bij wisselen.** Met een *data-join* (enter/update/exit) animeren de bestaande bellen soepel naar hun nieuwe plek en grootte als je van weergave wisselt — ze "morphen" in plaats van opnieuw te verschijnen.
- 5 **Inhoud van een bel.** Elke bel krijgt een kleurverloop, een glansplekje, het ingekleurde vis-plaatje (zie `fishImage.js`) en een label.
- 6 **Interactie.** Klik of hover op een bel → de info-regel toont de cijfers (bijv. "... kg biomassa (aantal × gewicht)"). De schakelknoppen wisselen de weergave.

```
// de drie manieren om de belgrootte te bepalen
const statFn = {
  count: d => d.count,
  weight: d => d.weight,
  biomass: d => d.count * d.weight, // aantal × gewicht
};
```

12 · Begrippenlijst — jargon uitgelegd

Term	In gewone taal
React-component	Een herbruikbaar stukje pagina, geschreven in <code>.jsx</code> (HTML met JavaScript erin). Levert de structuur en onthoudt toestand.
D3	Een bibliotheek voor datavisualisatie. Helpt vooral met <i>schalen</i> (data omzetten naar pixels) en het beheren van SVG-vormen.
SVG vs canvas	SVG = losse, schaalbare vormen (goed voor klikbare elementen). Canvas = één tekenvlak waarop je snel heel veel dingen tekent (goed voor tientallen bewegende visjes).
IntersectionObserver	Een browser-mechanisme dat meldt wanneer een element in of uit beeld scrolt. Hier gebruikt om grafieken pas te tekenen/animeren als je ze ziet.
requestAnimationFrame	Vraagt de browser om vóór elke schermverversing (~60x/sec) een functie te draaien. De basis van vloeiende animatie.
scaleSqrt (wortelschaal)	Zet een getal om naar een grootte via de wortel, zodat de <i>oppervlakte</i> eerlijk meeschaalt — twee keer zoveel = twee keer zo veel vlak, niet vier keer.
Force-simulatie	Een nep-natuurkunde van D3: elementen trekken/duwen elkaar tot ze een nette verdeling vinden (gebruikt bij het talenkoor).
Circle packing	Cirkels zo compact mogelijk in een vlak passen, op grootte (gebruikt bij het net).
Data-join (enter/update/exit)	D3-manier om data aan vormen te koppelen: nieuwe data → nieuwe vormen (enter), gewijzigde → bijwerken (update), verdwenen → weghalen (exit). Maakt soepele overgangen mogelijk.
Lazy loading	Iets pas doen wanneer het nodig is (hier: een grafiek pas tekenen als hij in beeld komt).
Seeded random	"Willekeur" met een vaste startwaarde, zodat je elke keer dezelfde reeks krijgt — handig voor reproduceerbare posities.
device pixel ratio	Hoeveel echte scherpixels één CSS-pixel beslaat (op retina-schermen 2x). Canvas wordt hierop afgesteld om scherp te blijven.
tinten (luminantie-behoudend)	Een grijze foto inkleuren door de helderheid te behouden en met een kleur te vermenigvuldigen — textuur blijft, kleur verandert.

13 · Bijlage — de `legacy/`-map

Beide mappen bevatten een submap `legacy/` met **oudere, niet meer gebruikte** versies van grafieken en hoofdstukken (o.a. de Hero-deeltjeswolk, de Ring-kalender, de Wereldkaart, en eerdere experimenten zoals `funnel`, `peaks`, `tide`, `shoal`).

Deze staan **niet** op de huidige Mitchell-pagina (in `Mitchell.jsx` zijn ze uitgecommentarieerd) en zijn daarom in deze gids buiten beschouwing gelaten. De werkende pagina bestaat uit de vijf besproken hoofdstukken: **Carrousel**, **Aquarium**, **Radar**, **Talen** en **Net**.

Wil je de legacy-bestanden er ook bij? Die kunnen op dezelfde manier worden uitgelegd; ze gebruiken dezelfde bouwstenen (state, D3, IntersectionObserver). Vraag het gerust.